

Exploration of the existence of binary linear $[2k, k, k]$ -codes and their properties

Julia Sikora, Elisaveta Bobrova, Justin Lin

April 18, 2024

1 Introduction

In the realm of modern technology, the storage and transmission of vast amounts of data is of importance. However, amidst these endeavors, the potential for errors during data storage can cause troubles. Even minor inaccuracies can lead to the failure of information reception or its incorrect interpretation.

To reduce such risks, coding theory emerges as a solution, offering mechanisms to introduce redundancy for the detection, correction, and reconstruction of errors in data transmission. One of the concepts that allows for the efficient transmission of information are binary linear codes.

This paper directs its focus towards exploring the properties of $[2k, k, k]$ -codes. These are the codes that have the rate and relative distance equal to $\frac{1}{2}$, so they provide an equal trade off between the efficiency and ability to correct mistakes while transmitting information. These properties alone should be enough to draw our attention to such codes, but more importantly, it turns out that it is possible to construct a lower bound on the minimal length of codewords of binary linear $[2k, k, k]$ codes. This can be achieved using the Griesmer bound created by mathematician John Griesmer in 1960. This concept allows us to determine existence of $[2k, k, k]$ code for some $k \in \mathbb{N}$.

2 Construction of elementary binary linear $[2k, k, k]$ -codes

It can be noted that the construction of elementary binary linear $[2k, k, k]$ codes for small k is not difficult. In this section examples of such codes are presented.

Each binary linear $[2k, k, k]$ -code C , is a subspace of the vector space $\mathbb{F}_2^{2k}$. Therefore, $(C, +, \mathbf{0})$ forms an abelian subgroup of $(\mathbb{F}_2^{2k}, +, \mathbf{0})$ under addition modulo 2. As a result, each code C needs to satisfy the following properties:

- for all $u \in C \setminus \{\mathbf{0}\}$, $u = (u_1, u_2, \dots, u_{2k})$, where $u_i \in \{0, 1\}$ for all $i \in \{1, 2, \dots, 2k\}$ and $|u| \geq d$;
- there exists k linearly independent words, $u^1, u^2, \dots, u^k \in C$, such that $\text{span}\{u^1, u^2, \dots, u^k\} = C$;
- $\#C = 2^k$.

When codes that satisfy aforementioned conditions are successfully constructed, we can elaborate on the equivalence relations between them. As C can be obtained from its generator matrix G by multiplying it with all words $v \in \mathbb{F}_2^k$, the linearly independent words, $u^1, u^2, \dots, u^k$ mentioned above are exactly the rows of the generator matrix. What is more, if C is obtained using the generator matrix, then it follows by construction that $\#C = 2^k$. Moreover, as the generator matrices lie in $M_{k \times 2k}(\mathbb{F}_2)$, we can check if the codes corresponding to these generator matrices are equivalent by transforming two generator matrices to their row

reduced echelon form, and then checking if a permutation of columns can lead to an equality of matrices. If this is the case, then codes are equivalent.

As a result, we can write a program¹ that constructs $[2k, k, k]$ -codes.

- First, we construct a list L with all possible words of length $2k$ and next remove all of the words with weight less than k .
- Secondly, we will pick the first word in L , $u^1 = u^{1_1}$ and find another word $u^{1_2} \in L$ such that $|u^{1_1} + u^{1_2}| \geq k$. Similarly, we find another word $u^{1_3} \in L$ such that we have $|u^{1_1} + u^{1_3}|, |u^{1_2} + u^{1_3}|, |u^{1_1} + u^{1_2} + u^{1_3}| \geq k$. We will repeat this process, until we obtain set of words $u^{1_1}, u^{1_2}, \dots, u^{1_k}$. What is more, there needs to exist $i \in \{1, 2, \dots, k\}$ such that $|u^{1_i}| = k$, as the minimum weight should equal k . Otherwise, it would not be a valid $[2k, k, k]$ -code. Using previously found words, we can construct the generator matrix of this code, which is then transform into its row reduced echelon form.
- We repeat the previous steps for all $u^1, u^2, \dots, u^{\#L} \in L$, such that we obtain a set of generator matrices M .
- Next, we take the first generator matrix, G_1 in M , and check if it can be obtained from another generator matrix, G_j in M , by a permutation of columns. If that is the case, we will remove G_j from M . Afterwards, we take the next generator matrix G_2 and apply the same procedure. As a result, our set M only consists of generator matrices that correspond to non-equivalent codes.
- Finally, to construct these codes, we use the definition of the generator matrix such that $C = \{v \cdot G : v \in \mathbb{F}_2^k\}$.

Now we can use this program to construct a few examples of $[2k, k, k]$ -codes for $k = 1, 2, 3, 4$.

2.1 $[2, 1, 1]$

For $k = 1$ we claim that

$$C = \{(00), (01)\}$$

is a suitable $[2, 1, 1]$ -code.

Proof.

- For all words $u \in C$ we have that $u \in \mathbb{F}_2^2$ and if $u \neq \mathbf{0}$ then $|u| \geq 1$;
- there exists a word $(01) \in C$ with
$$\text{span}\{(01)\} = C;$$
- $\#C = 2$.

So indeed this is an example of binary linear $[2, 1, 1]$ code. □

2.2 $[4, 2, 2]$

For $k = 2$ we claim that

$$C = \{(0000), (0111), (1001), (1110)\}$$

is a suitable $[4, 2, 2]$ -code.

Proof.

¹The programs for the cases $k = 1, 2, 3, 4$ are presented in the appendix. Running the programs, gives the codes, which are not equivalent to one another.

- For all words $u \in C$ we have that $u \in \mathbb{F}_2^4$ and if $u \neq \mathbf{0}$ then $|u| \geq 2$;
- there exists two linearly independent words $(0111), (1001) \in C$ with

$$\text{span}\{(0111), (1001)\} = C;$$

- $\#C = 4 = 2^2$.

Which demonstrates that this code is an example of a binary linear $[4, 2, 2]$ code. $\square$

2.3 $[6, 3, 3]$

For $k = 3$ we claim that

$$C = \{(000000), (001101), (010111), (011010), (100011), (101110), (110100), (111001)\}$$

is a suitable $[6, 3, 3]$ -code.

Proof.

- For all words $u \in C$ we have that $u \in \mathbb{F}_2^6$ and if $u \neq \mathbf{0}$ then $|u| \geq 3$;
- there exists three linearly independent words $(001101), (010111), (011010) \in C$ with

$$\text{span}\{(001101), (010111), (011010)\} = C;$$

- $\#C = 8 = 2^3$.

Which shows that this code is a binary linear $[6, 3, 3]$ code. $\square$

2.4 $[8, 4, 4]$

For $k = 4$ we claim that

$$C = \left\{ (00000000), (00001111), (00110011), (01100101), (10100110), (00111100), (01101010), (10101001), \right. \\ \left. (01010110), (10010101), (11000011), (01011001), (10011010), (11001100), (11110000), (11111111) \right\}$$

is a suitable $[8, 4, 4]$ -code.

Proof.

- For all words $u \in C$ we have that $u \in \mathbb{F}_2^8$ and if $u \neq \mathbf{0}$ then $|u| \geq 4$;
- there exists four linearly independent words $(00001111), (00110011), (01100101), (10100110) \in C$ with

$$\text{span}\{(00001111), (00110011), (01100101), (10100110)\} = C;$$

- $\#C = 16 = 2^4$.

Hence, this is a suitable $[8, 4, 4]$ -code. $\square$

3 Griesmer bound

As it was mentioned before Griesmer bound provides a way to compute a lower bound on the length of the codewords in a code of dimension k and minimal distance d . Even though in this paper only the binary case is considered, this result can be easily generalized to non-binary codes by replacing 2 with some number $q \in \mathbb{N}$, where q is the length of alphabet used to create a code. In this paper the proof for binary case is provided, but the result for more general case follows in a similar manner.

Suppose we have a binary linear code $[n, k, d] \in \mathbb{F}_2$, where coefficients n, k, d represent the length, dimension and minimum distance respectively. Then the Griesmer bound for binary linear codes is

$$n \geq \sum_{i=0}^{k-1} \left\lceil \frac{d}{2^i} \right\rceil.$$

Proof. We begin by considering some binary linear code C of dimension k , minimal distance d and minimal length $n = N(k, d)$. We want to show that

$$N(k, d) \geq \sum_{i=0}^{k-1} \left\lceil \frac{d}{2^i} \right\rceil$$

Without loss of generality we may suppose that its generator matrix of C is of the form:

$$G = \begin{bmatrix} 1 & \cdots & 1 & 0 & \cdots & 0 \\ \star & \star & \star & & G' & \end{bmatrix}$$

where matrix G is of dimension $d \times N(k, d)$. The first row of G is of form $r = (1, \dots, 1, 0, \dots, 0)$, which is a vector starting with d ones followed by $N(k, d) - d$ zeros, so r is of weight d .

Now we can notice that matrix G' is a generator matrix of some binary linear code C' of dimension $k' = k - 1$ and length $n' = N(k, d) - d$. Let the minimal distance of C' be denoted as d' , so that $n' = N(k', d')$ is its length. Note that codes whose generator matrix is constructed in this manner are called residual codes.

We choose some $g \in C'$ such that $w(g) = d'$ (where $w(g) = |g|$). We know that words of such weight exists in C' by the definition of minimal distance. We also consider a vector $h \in \mathbb{F}_2^d$ such that the concatenation $(h|g) \in C$. Note that we are able to obtain such a concatenation since g is either one of the rows of G' or linear combination of them, namely $g \in \mathbb{F}_2^{N(k, d) - d}$, so appending it with an appropriately chosen $h \in \mathbb{F}_2^d$ results in a vector $(h|g) \in \mathbb{F}_2^{N(k, d)}$ in C .

We observe that

$$w(h) + w(g) = w(h|g) \geq d \tag{1}$$

since $(h|g) \in C$ and d is a minimal weight of a word in C . Furthermore, given that $r \in C$ and C is linear, we have $(h|g) + r \in C$ and $w((h|g) + r) \geq d$. Having that h is of length d , and first d entries of r are ones, we can rewrite it as $w((h|g) + r) = w(((1, \dots, 1) + h)|g)$. It follows that

$$w(((1, \dots, 1) + h)|g) = w((1, \dots, 1) + h) + w(g) = d - w(h) + w(g) \geq d \tag{2}$$

Combining (1) and (2) we obtain $d + 2w(g) \geq 2d$. Recalling that we chose g in a way that $w(g) = d'$, we get that $d' \geq \lceil \frac{d}{2} \rceil$. We use the ceiling function here because d' is an integer and to avoid underestimation of the sum.

The aforementioned definition $n' = N(k', d')$ and the result $d' \geq \lceil \frac{d}{2} \rceil$ imply that the length of the code C' can be denoted as $n' \geq N(k - 1, \lceil \frac{d}{2} \rceil)$. Recalling that $n' = N(k, d) - d$ we obtain the inequality

$$N(k, d) \geq d + N(k-1, \lceil \frac{d}{2} \rceil).$$

To obtain the final result we perform induction on k .

Base case. Let $k = 1$. For an $[n, 1, d]$ code we have

$$N(1, d) \geq d + N(0, \lceil \frac{d}{2} \rceil) = d + 0 = d = \sum_{i=0}^0 \lceil \frac{d}{2^i} \rceil$$

as required.

Induction hypothesis. For $k > 1$ we have that $n' = N(k-1, d') \geq \sum_{i=0}^{k-2} \lceil \frac{d'}{2^i} \rceil$ holds for the residue code C' $[n-d, k-1, d']$, so a code of dimension $k-1$. We want to show that it also holds for the code of dimension k .

Induction step. We begin by noting that $n' = N(k, d) - d = n - d$, so using the induction hypothesis we have that

$$n' = n - d \geq \sum_{i=0}^{k-2} \lceil \frac{d'}{2^i} \rceil \geq \sum_{i=0}^{k-2} \lceil \frac{\lceil \frac{d}{2} \rceil}{2^i} \rceil = \sum_{i=0}^{k-2} \lceil \frac{d}{2^{i+1}} \rceil$$

where second inequality follows from the fact that $d' \geq \lceil \frac{d}{2} \rceil$ and the last equality follows from the identity $\lceil \frac{\lceil \frac{a}{2^j} \rceil}{2} \rceil = \lceil \frac{a}{2^{j+1}} \rceil$.

Now we can rewrite it in a form

$$n \geq d + \sum_{i=0}^{k-2} \lceil \frac{d}{2^{i+1}} \rceil = d + \sum_{i=1}^{k-1} \lceil \frac{d}{2^i} \rceil = \sum_{i=0}^{k-1} \lceil \frac{d}{2^i} \rceil$$

and by the manipulation of sums we get the desired result that $n = N(k, d) \geq \sum_{i=0}^{k-1} \lceil \frac{d}{2^i} \rceil$. □

This proof was adapted from the article written by John Griesmer titled "A bound for error-correcting codes". [1]

The Griesmer bound, therefore, identifies the smallest integer n for which a binary $[n, k, d]$ -code exists. It follows immediately that we can confirm the existence of the binary $[2, 1, 1]$, $[4, 2, 2]$, $[6, 3, 3]$, $[8, 4, 4]$ codes by showing that they attain the Griesmer bound:

$$n(1, 1) \geq \sum_{i=0}^0 \lceil \frac{1}{2^i} \rceil = \lceil \frac{1}{1} \rceil = 1,$$

$$n(2, 2) \geq \sum_{i=0}^1 \lceil \frac{2}{2^i} \rceil = \lceil \frac{2}{1} \rceil + \lceil \frac{2}{2} \rceil = 2 + 1 = 3,$$

$$n(3, 3) \geq \sum_{i=0}^2 \lceil \frac{3}{2^i} \rceil = \lceil \frac{3}{1} \rceil + \lceil \frac{3}{2} \rceil + \lceil \frac{3}{4} \rceil = 3 + 2 + 1 = 6,$$

$$n(4, 4) \geq \sum_{i=0}^3 \lceil \frac{4}{2^i} \rceil = \lceil \frac{4}{1} \rceil + \lceil \frac{4}{2} \rceil + \lceil \frac{4}{4} \rceil + \lceil \frac{4}{8} \rceil = 4 + 2 + 1 + 1 = 8$$

as expected.

On the other hand, observe that

$$n(5, 5) \geq \sum_{i=0}^4 \left\lceil \frac{5}{2^i} \right\rceil = \left\lceil \frac{5}{1} \right\rceil + \left\lceil \frac{5}{2} \right\rceil + \left\lceil \frac{5}{4} \right\rceil + \left\lceil \frac{5}{8} \right\rceil + \left\lceil \frac{5}{16} \right\rceil = 5 + 3 + 2 + 1 + 1 = 12.$$

Clearly, $10 \not\geq 12$, hence the $[10, 5, 5]$ -code does not satisfy the bound, implying that such code cannot exist.

4 Table of the bounds on the minimum distance of an $[n, k, d]$ code

We use <http://www.codetables.de/> to construct a table of bounds on the minimum distance of linear codes $[n, k, d]$ over $\mathbb{F}_2$. The table obtained from this website is constructed using the Brouwer's table (as of 2007-02-13). The upper bounds mainly follow from the Griesmer bound by considering the possibilities of d in order for the bound to be attained for a chosen n , although for greater values such as $[n, k] = [60, 30]$ the upper bound is found by considering a shortening of the code, which is out of the scope of our paper.

Let us consider a table of upper bounds on the distance d for $1 \leq n \leq 20$ and $1 \leq k \leq 10$:

n/k	1	2	3	4	5	6	7	8	9	10
1	1									
2	2	1								
3	3	2	1							
4	4	2	2	1						
5	5	3	2	2	1					
6	6	4	3	2	2	1				
7	7	4	4	3	2	2	1			
8	8	5	4	4	2	2	2	1		
9	9	6	4	4	3	2	2	2	1	
10	10	6	5	4	4	3	2	2	2	1
11	11	7	6	5	4	4	3	2	2	2
12	12	8	6	6	4	4	4	3	2	2
13	13	8	7	6	5	4	4	4	3	2
14	14	9	8	7	6	5	4	4	4	3
15	15	10	8	8	7	6	5	4	4	4
16	16	10	8	8	8	6	6	5	4	4
17	17	11	9	8	8	7	6	6	5	4
18	18	12	10	8	8	8	7	6	6	4
19	19	12	10	9	8	8	8	7	6	5
20	20	13	11	10	9	8	8	8	7	6

Table 1: Bounds on the minimum distance of linear codes over $\mathbb{F}_2$

Choosing a value for k and looking at the corresponding upper bound of the minimum distance for $n = 2k$, we observe the following:

- for $k = 1$ and $n = 2$, $d \leq 2$,
- for $k = 2$ and $n = 4$, $d \leq 2$,
- for $k = 3$ and $n = 6$, $d \leq 3$,

- for $k = 4$ and $n = 8$, $d \leq 4$,

This successfully agrees with the existence of the $[2k, k, k]$ -codes for $k = 1, 2, 3, 4$.

Moreover, if we look further into other combinations of k and $n = 2k$ in our table, we see that:

- for $[10, 5, 5]$ the minimum distance is at most 4,
- for $[12, 6, 6]$ the minimum distance is at most 4,
- for $[14, 7, 7]$ minimum distance is at most 4,
- for $[16, 8, 8]$ minimum distance is at most 5,
- for $[18, 9, 9]$ the minimum distance is at most 6,
- for $[20, 10, 10]$ the minimum distance is at most 6.

Noticeably, in all of these cases the minimum distance is less than the value of k . This observation leads us to the following claim:

Claim. For $k > 4$ we have that $n(k, d) = 2k < \sum_{i=0}^{k-1} \left\lceil \frac{d}{2^i} \right\rceil$. Which means that the Griesmer bound is not satisfied and from this we can deduce that for a $[2k, k, k]$ -code to exist k must be at most equal to 4.

Proof. We prove the claim by induction. Let $k \geq 5$.

Base step: For $k = 5$,

$$\sum_{i=0}^4 \left\lceil \frac{5}{2^i} \right\rceil = \left\lceil \frac{5}{1} \right\rceil + \left\lceil \frac{5}{2} \right\rceil + \left\lceil \frac{5}{4} \right\rceil + \left\lceil \frac{5}{8} \right\rceil + \left\lceil \frac{5}{16} \right\rceil = 5 + 3 + 2 + 1 + 1 = 12 > 10.$$

So base case is satisfied.

Induction hypothesis: Given $k \geq 5$ it holds that $2k < \sum_{i=0}^{k-1} \left\lceil \frac{k}{2^i} \right\rceil$.

Induction step: For $k + 1$

$$\begin{aligned} \sum_{i=0}^k \left\lceil \frac{k+1}{2^i} \right\rceil &= \sum_{i=0}^{k-1} \left\lceil \frac{k+1}{2^i} \right\rceil + \left\lceil \frac{k+1}{2^k} \right\rceil \\ &= \sum_{i=1}^{k-1} \left\lceil \frac{k+1}{2^i} \right\rceil + k + 1 + \left\lceil \frac{k+1}{2^k} \right\rceil \\ &> 2k + 1 + 1 = 2k + 2 = 2(k + 1) \end{aligned} \tag{3}$$

where the inequality follows from the induction hypothesis, as well as the fact that $\left\lceil \frac{k+1}{2^i} \right\rceil \geq \left\lceil \frac{k}{2^i} \right\rceil$ and that $\left\lceil \frac{k+1}{2^k} \right\rceil = 1$. □

5 Conclusion

$[2k, k, k]$ binary linear codes are mathematical objects that can be explored and understood using basic techniques from coding theory. However, as it observed in this report, the existence of such codes faces limitations, namely, these codes cannot be constructed for $k \geq 5$ due to the nature of the Griesmer bound. Although this restricts the practical use of $[2k, k, k]$ codes, they can be an entry way into more advanced coding theory and constructions of more complex codes.

6 Appendix

For the following program, notice that for each k , only the loop where we find the linear independent vectors that span the generator matrix, differ. The rest of the code is exactly the same.

For $k = 1$:

```
import numpy as np
import sympy

def decomposition_in_powers_of_two (r):
    list = []
    while r != 0:
        i = 0
        while 2**i <= r:
            i += 1
        i -= 1
        r -= 2**i
        list.append(i)
    return list

def words_in_F(k):
    words = []
    for i in range(1, 2 ** (k)):
        list = decomposition_in_powers_of_two(i)
        binary = [0] * k
        for j in list:
            binary[j] += 1
        words.append(binary[: -1])
    words.insert(0, [0] * k)
    return words

def addition(word_1, word_2):
    word_1 = np.array(word_1)
    word_2 = np.array(word_2)
    word = word_1 + word_2
    for i in range(word.size):
        if word[i] == 2:
            word[i] = 0
    return word.tolist()

def codes(k):
    # Constructing list of words with length 2k and weight > k - 1
    list_weight_geq_k = []
    for word in words_in_F(2 * k):
        if sum(word) >= k:
            list_weight_geq_k.append(word)

    # Constructing the generator matrices
    list = []
    for a in list_weight_geq_k:
```



```

    if sum(a) == k:
        list.append([a])

generator_matrices = np.absolute(np.array(list))
generator_matrices = generator_matrices.tolist()

# Equivalence between generator matrices
representative_set = []
while len(generator_matrices) != 0:
    delete = []
    representative = generator_matrices[0]
    representative_set.append(representative)
    for i in range(len(generator_matrices)):
        if sorted(np.array(representative).transpose().tolist())
           == sorted(np.array(generator_matrices[i].transpose().tolist())):
            delete.append(i)
    for j in delete[::-1]:
        del generator_matrices[j]
    representative_set = np.array(representative_set)

# Computing the codes
list_of_codes=[]
words = np.array(words_in_F(k))
for representative in representative_set:
    code = []
    for word in words:
        code.append(representative.transpose().dot(word).transpose())
    list_of_codes.append(np.array(code))

list_of_codes = np.array(list_of_codes)

for i in range(len(list_of_codes)):
    for j in range(2 ** k):
        for l in range(2 * k):
            if list_of_codes[i][j][1] % 2 == 0:
                list_of_codes[i][j][1] = 0
            else:
                list_of_codes[i][j][1] = 1
return np.array(list_of_codes)

list = codes(1).tolist()
for i in range(len(list)):
    print(f'C_{i + 1} = {list[i]}')

# COMPUTATION TIME < 1 SECONDS

k = 2:

import numpy as np
import sympy

```

```

def decomposition_in_powers_of_two (r):
    list = []
    while r != 0:
        i = 0
        while 2**i <= r:
            i += 1
        i -= 1
        r -= 2**i
        list.append(i)
    return list

def words_in_F(k):
    words = []
    for i in range(1, 2 ** (k)):
        list = decomposition_in_powers_of_two(i)
        binary = [0] * k
        for j in list:
            binary[j] += 1
        words.append(binary[: -1])
    words.insert(0, [0] * k)
    return words

def addition(word_1, word_2):
    word_1 = np.array(word_1)
    word_2 = np.array(word_2)
    word = word_1 + word_2
    for i in range(word.size):
        if word[i] == 2:
            word[i] = 0
    return word.tolist()

def codes(k):

    # Constructing list of words with length 2k and weight > k - 1
    list_weight_geq_k = []
    for word in words_in_F(2 * k):
        if sum(word) >= k:
            list_weight_geq_k.append(word)

    # Computing the generator matrices
    list = []
    for a in list_weight_geq_k:
        for b in list_weight_geq_k:
            a_b = addition(a, b)
            if sum(a_b) >= k:
                for q in [a, b, a_b]:
                    if sum(q) == k:
                        list.append(sympy.Matrix(np.array([a, b])).rref()[0])
                        break

    generator_matrices = np.absolute(np.array(list))

```

```

generator_matrices = generator_matrices.tolist()

# Equivalence between generator matrices
representative_set = []
while len(generator_matrices) != 0:
    delete = []
    representative = generator_matrices[0]
    representative_set.append(representative)
    for i in range(len(generator_matrices)):
        if sorted(np.array(representative).transpose().tolist())
           == sorted(np.array(generator_matrices[i].transpose().tolist())):
            delete.append(i)
    for j in delete[::-1]:
        del generator_matrices[j]
    representative_set = np.array(representative_set)

# Computing the codes
list_of_codes=[]
words = np.array(words_in_F(k))
for representative in representative_set:
    code = []
    for word in words:
        code.append(representative.transpose().dot(word).transpose())
    list_of_codes.append(np.array(code))

list_of_codes = np.array(list_of_codes)

for i in range(len(list_of_codes)):
    for j in range(2 ** k):
        for l in range(2 * k):
            if list_of_codes[i][j][1] % 2 == 0:
                list_of_codes[i][j][1] = 0
            else:
                list_of_codes[i][j][1] = 1
return np.array(list_of_codes)

list = codes(2).tolist()
for i in range(len(list)):
    print(f'C_{i + 1} = {list[i]}')

# COMPUTATION TIME = ~1 SECONDS

k = 3:

import numpy as np
import sympy

def decomposition_in_powers_of_two (r):
    list = []
    while r != 0:
        i = 0

```

```

    while 2**i <= r:
        i += 1
    i -= 1
    r -= 2**i
    list.append(i)
return list

def words_in_F(k):
    words = []
    for i in range(1, 2 ** (k)):
        list = decomposition_in_powers_of_two(i)
        binary = [0] * k
        for j in list:
            binary[j] += 1
        words.append(binary[: -1])
    words.insert(0, [0] * k)
    return words

def addition(word_1, word_2):
    word_1 = np.array(word_1)
    word_2 = np.array(word_2)
    word = word_1 + word_2
    for i in range(word.size):
        if word[i] == 2:
            word[i] = 0
    return word.tolist()

def codes(k):

    # Constructing list of words with length 2k and weight > k - 1
    list_weight_geq_k = []
    for word in words_in_F(2 * k):
        if sum(word) >= k:
            list_weight_geq_k.append(word)

    # Computing the generator matrices
    list = []
    for a in list_weight_geq_k:
        for b in list_weight_geq_k:
            a_b = addition(a, b)
            if sum(a_b) >= k:
                for c in list_weight_geq_k:
                    a_c = addition(a, c)
                    b_c = addition(b, c)
                    a_b_c = addition(a_b, c)
                    if sum(a_c) >= k and sum(b_c) >= k and sum(a_b_c) >= k:
                        for q in [a, b, c, a_b, a_c, b_c, a_b_c]:
                            if sum(q) == k:
                                list.append(sympy.Matrix(np.array([a, b, c])).rref()[0])
                                break

```

```

generator_matrices = np.absolute(np.array(list))
generator_matrices = generator_matrices.tolist()

# Equivalence between generator matrices
representative_set = []
while len(generator_matrices) != 0:
    delete = []
    representative = generator_matrices[0]
    representative_set.append(representative)
    for i in range(len(generator_matrices)):
        if sorted(np.array(representative).transpose().tolist())
           == sorted(np.array(generator_matrices[i].transpose().tolist())):
            delete.append(i)
    for j in delete[::-1]:
        del generator_matrices[j]
    representative_set = np.array(representative_set)

# Computing the codes
list_of_codes = []
words = np.array(words_in_F(k))
for representative in representative_set:
    code = []
    for word in words:
        code.append(representative.transpose().dot(word).transpose())
    list_of_codes.append(np.array(code))

list_of_codes = np.array(list_of_codes)

for i in range(len(list_of_codes)):
    for j in range(2 ** k):
        for l in range(2 * k):
            if list_of_codes[i][j][1] % 2 == 0:
                list_of_codes[i][j][1] = 0
            else:
                list_of_codes[i][j][1] = 1
return np.array(list_of_codes)

list = codes(3).tolist()
for i in range(len(list)):
    print(f'C_{i + 1} = {list[i]}')

# COMPUTATION TIME = ~1230 SECONDS

k = 4:

import numpy as np
import sympy

def decomposition_in_powers_of_two (r):
    list = []
    while r != 0:

```

```

    i = 0
    while 2**i <= r:
        i += 1
    i -= 1
    r -= 2**i
    list.append(i)
return list

def words_in_F(k):
    words = []
    for i in range(1, 2 ** (k)):
        list = decomposition_in_powers_of_two(i)
        binary = [0] * k
        for j in list:
            binary[j] += 1
        words.append(binary[::-1])
    words.insert(0, [0] * k)
    return words

def addition(word_1, word_2):
    word_1 = np.array(word_1)
    word_2 = np.array(word_2)
    word = word_1 + word_2
    for i in range(word.size):
        if word[i] == 2:
            word[i] = 0
    return word.tolist()

def codes(k):

    # Constructing list of words with length 2k and weight > k - 1
    list_weight_geq_k = []
    for word in words_in_F(2 * k):
        if sum(word) >= k:
            list_weight_geq_k.append(word)

    # Computing the generator matrices
    list = []
    for a in list_weight_geq_k:
        for b in list_weight_geq_k:
            a_b = addition(a, b)
            if sum(a_b) >= k:
                for c in list_weight_geq_k:
                    a_c = addition(a, c)
                    b_c = addition(b, c)
                    a_b_c = addition(a_b, c)
                    if sum(a_c) >= k and sum(b_c) >= k and sum(a_b_c) >= k:
                        for d in list_weight_geq_k:
                            a_d = addition(a, d)
                            b_d = addition(b, d)
                            c_d = addition(c, d)

```

```

a_b_d = addition(a_b, d)
a_c_d = addition(a_c, d)
b_c_d = addition(b_c, d)
a_b_c_d = addition(a_b_c, d)
if sum(a_d) >= k and sum(b_d) >= k and sum(c_d) >= k and sum(a_b_d)
    >= k and sum(a_c_d) >= k and sum(b_c_d) >= k and sum(a_b_c_d) >= k:
    for q in [a, b, c, d, a_b, a_c, a_d, b_c, b_d, c_d, a_b_c, a_b_d,
        a_c_d, b_c_d, a_b_c_d]:
        if sum(q) == k:
            list.append(sympy.Matrix(np.array([a, b, c, d])).rref()[0])
            break

generator_matrices = np.absolute(np.array(list))
generator_matrices = generator_matrices.tolist()

# Equivalence between generator matrices
representative_set = []
while len(generator_matrices) != 0:
    delete = []
    representative = generator_matrices[0]
    representative_set.append(representative)
    for i in range(len(generator_matrices)):
        if sorted(np.array(representative).transpose().tolist())
            == sorted(np.array(generator_matrices[i].transpose().tolist())):
            delete.append(i)
    for j in delete[::-1]:
        del generator_matrices[j]
    representative_set = np.array(representative_set)

# Computing the codes
list_of_codes = []
words = np.array(words_in_F(k))
for representative in representative_set:
    code = []
    for word in words:
        code.append(representative.transpose().dot(word).transpose())
    list_of_codes.append(np.array(code))

list_of_codes = np.array(list_of_codes)

for i in range(len(list_of_codes)):
    for j in range(2 ** k):
        for l in range(2 * k):
            if list_of_codes[i][j][l] % 2 == 0:
                list_of_codes[i][j][l] = 0
            else:
                list_of_codes[i][j][l] = 1
return np.array(list_of_codes)

list = codes(4).tolist()

```

```
for i in range(len(list)):
    print(f'C_{i + 1} = {list[i]}')

# COMPUTATION TIME > 432000 SECONDS
```

References

- [1] J.H. Griesmer. “A bound for error-correcting codes”. In: *IBM Journal of Res. and Dev* (1960).